Arborescence d'une application web

Une application web Java déployée sur Tomcat suit une structure de dossiers précise, que Tomcat sait lire. Elle se place dans le dossier `webapps`, dans un dossier qui porte le nom du projet (c'est le `<project-name>` qu'on retrouve dans les URL).

```
webapps/
'-- <project-name>/
    |-- *.html, *.jsp          <- pages accessibles depuis le navigateur
    '-- WEB-INF/               <- non accessible depuis le navigateur
        |-- web.xml            <- descripteur de deploiement
        |-- classes/           <- classes compilees (.class)
        |   '-- <package>/     <- un dossier par package Java
        '-- lib/               <- bibliotheques (.jar) de l'application
```

À la racine du projet, on met les pages HTML et JSP : elles sont accessibles directement, par exemple avec `http://localhost:8080/<project-name>/index.html`. Le dossier `WEB-INF` est protégé : le navigateur ne peut pas y accéder, et c'est là que Tomcat cherche ce dont il a besoin. Il contient le fichier `web.xml`, qui sert à déclarer les servlets et leurs URL, le dossier `classes`, qui contient les classes compilées, et le dossier `lib`, pour les fichiers `.jar` dont l'application a besoin. Dans `classes`, il faut respecter les packages : une classe du package `ex1` doit être dans `WEB-INF/classes/ex1`, sinon Tomcat ne la trouve pas.

Dans ce TP, cette structure est utilisée pour les trois parties : les servlets (partie 1) sont dans le package `ex1`, les pages JSP (partie 2) sont à la racine du projet, et le bean ainsi que le modèle et le contrôleur du MVC (partie 3) sont dans leurs packages respectifs sous `WEB-INF/classes`.

```
C:\Program Files\Apache Software Foundation\Tomcat 11.0\webapps\TP1>tree /F
Structure du dossier
Le numéro de série du volume est 6CA2-98EB
C:.
|   index.html
|   items.html
|
└───WEB-INF
    |   web.xml
    |
    └───classes
        └───ex1
                FirstServlet.class
                FirstServlet.java
                ListItems.class
                ListItems.java
                ReadCookie.class
                ReadCookie.java
                ShowSession.class
                ShowSession.java
                VisitCounter.class
                VisitCounter.java
```

Figure 1 – Arborescence réelle du projet, affichée avec la commande `tree /F`.

Exercice 1 – Java Servlet

Dans cet exercice, on utilise des servlets pour récupérer les données d'un formulaire, puis pour les garder avec des cookies et avec une session. Le code complet de tous les servlets est dans mon dépôt GitHub (`\repourl`). Ici je montre seulement les passages importants.

1.1 – Traitement d'une requête POST

1- Question 1 : Formulaire HTML et servlet

Le formulaire envoie le prénom, le nom et la date de naissance en POST au servlet `FirstServlet`. Dans le servlet, chaque valeur est lue avec `getParameter()`. Il faut utiliser exactement le même nom que l'attribut `name` du champ HTML, sinon on récupère `null`.

```
<form action="FormServlet" method="POST">
  <input type="text" name="firstName">
  <input type="text" name="lastName">
  <input type="text" name="birthDay" placeholder="mm/dd/yyyy">
  <input type="submit">
</form>
```

```
String firstName = request.getParameter("firstName");
String lastName = request.getParameter("lastName");
String birthDay = request.getParameter("birthDay");
```

Ensuite le servlet écrit une page HTML avec les valeurs reçues grâce à `response.getWriter()`.



Figure 2 – Formulaire rempli et page affichée par `FirstServlet`.

Erreur rencontrée

Ma première compilation avec `javac` a échoué avec l'erreur *package jakarta.servlet does not exist*. Les classes des servlets ne sont pas dans le JDK : elles se trouvent dans le fichier `servlet-api.jar` du dossier `lib` de Tomcat, et il faut l'ajouter au classpath avec l'option `-cp`. Les imports commencent par `jakarta` (et non `javax`) parce que la version de Tomcat est la 10 ou plus. Comme mes classes sont dans le package `ex1`, les fichiers `.class` doivent aussi être placés dans `WEB-INF/classes/ex1`.

2- Question 2 : Ajout du sexe

J'ai ajouté deux boutons radio avec le même `name`. C'est ce qui permet de n'en choisir qu'un seul, et la `value` du bouton coché est envoyée au servlet.

```
<input type="radio" name="sex" value="Male"> Male
<input type="radio" name="sex" value="Female"> Female
```

Dans le servlet, on lit ce paramètre avec `request.getParameter("sex")` et on l'affiche avec les autres informations.

3- Question 3 : URL personnalisées

Pour accéder au même servlet par `FormServlet` et par `Form_Servlet`, j'ai déclaré le servlet dans le fichier `web.xml` avec deux `servlet-mapping` qui pointent vers le même nom de servlet.

```
<servlet>
  <servlet-name>FirstServlet</servlet-name>
  <servlet-class>ex1.FirstServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>FirstServlet</servlet-name>
  <url-pattern>/FormServlet</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>FirstServlet</servlet-name>
  <url-pattern>/Form_Servlet</url-pattern>
</servlet-mapping>
```

Remarque : si on tape directement l'adresse du servlet dans le navigateur, on obtient une erreur 405. Le navigateur envoie alors une requête GET, alors que `FirstServlet` ne définit que `doPost()`.

1.2 – Gestion des cookies

1- Question 1 : Création des cookies

On modifie `FirstServlet` pour créer un cookie par champ, puis on les envoie au navigateur avec `addCookie()`.

```
Cookie c1 = new Cookie("firstName", firstName);
Cookie c2 = new Cookie("lastName", lastName);
Cookie c3 = new Cookie("birthDay", birthDay);
Cookie c4 = new Cookie("sex", sex);

response.addCookie(c1);
response.addCookie(c2);
response.addCookie(c3);
response.addCookie(c4);
```

Les cookies sont envoyés dans les en-têtes de la réponse HTTP, donc ce code doit être placé avant d'écrire la page avec `getWriter()`. Je n'ai pas utilisé `setMaxAge()` ici : ce sont donc des cookies de session, qui disparaissent quand on ferme le navigateur.

2- Question 2 : Vérification dans le navigateur

Après l'envoi du formulaire, on retrouve les quatre cookies dans les outils de développement du navigateur (onglet *Application*, rubrique *Cookies*).

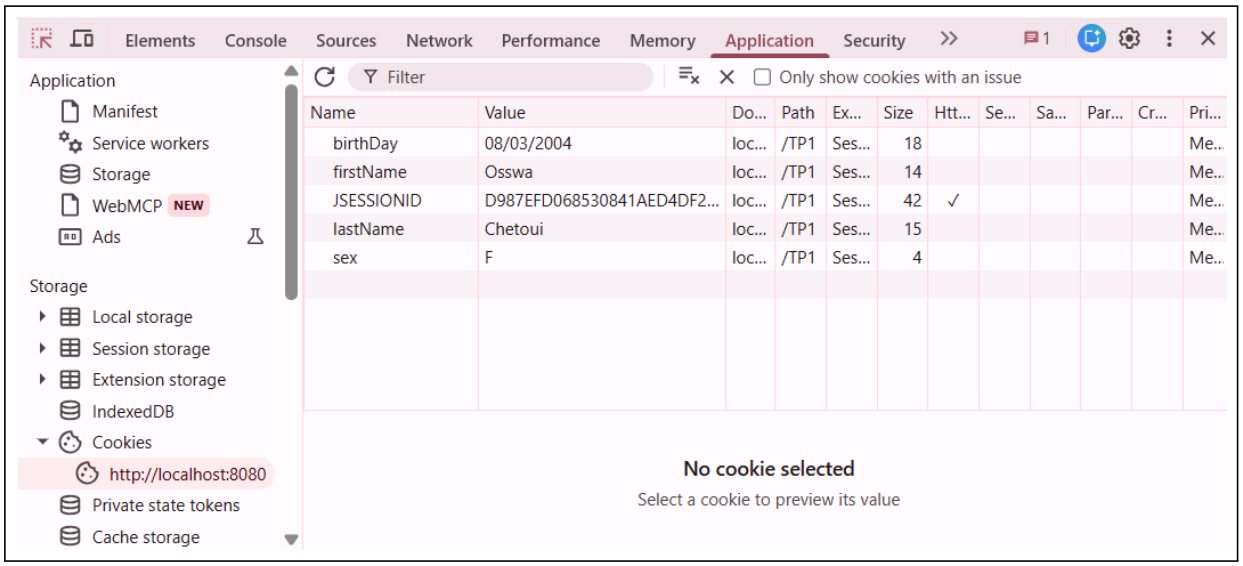


Figure 3 – Les quatre cookies créés par le servlet.

3- Question 3 : Servlet `ReadCookie`

Le servlet lit les cookies avec `request.getCookies()` et affiche seulement les quatre qui nous intéressent. On y accède par un lien, donc c'est une requête GET et j'ai écrit `doGet()`. Le lien vers `ReadCookie` est dans la page affichée par `FirstServlet`.

```
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie cookie : cookies) {
        String name = cookie.getName();
        if (name.equals("firstName") || name.equals("lastName")
                || name.equals("birthDay") || name.equals("sex")) {
            out.println("<p>" + name + " : " + cookie.getValue() + "</p>");
        }
    }
}
```

Il faut tester `cookies != null` car `getCookies()` renvoie `null` (et pas un tableau vide) quand le navigateur n'envoie aucun cookie. Le filtre sur les quatre noms est utile parce que Tomcat ajoute aussi son propre cookie `JSESSIONID`.

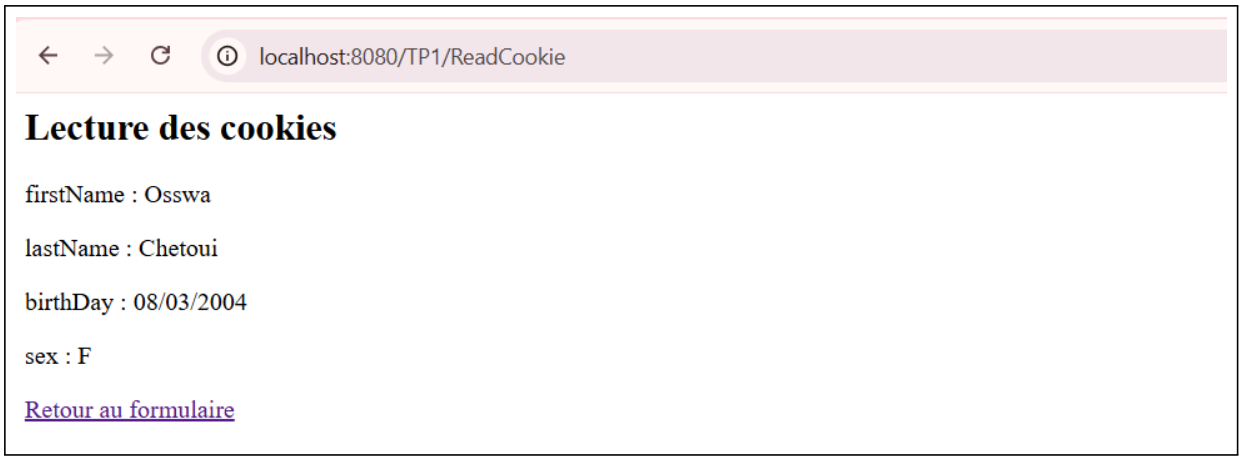


Figure 4 – Valeurs affichées par `ReadCookie`.

4- Question 4 : Extra : compteur de visites

Le servlet `VisitCounter` lit le cookie `visits`, ajoute 1 et le renvoie avec une durée de vie de 30 jours.

```
visits++;
Cookie visitCookie = new Cookie("visits", String.valueOf(visits));
visitCookie.setMaxAge(30 * 24 * 60 * 60);
response.addCookie(visitCookie);
```

La valeur d'un cookie est toujours du texte, donc on la convertit avec `Integer.parseInt()` (dans un `try/catch` au cas où la valeur ne serait pas un nombre). Le compteur est gardé chez le client : si on supprime les cookies, il repart de zéro.

1.3 – Suivi des sessions HTTP

1- Question 1 : Valeurs simples : `ShowSession`

Ici les données sont gardées côté serveur dans l'objet `HttpSession`. Le navigateur ne garde que l'identifiant de session (le cookie `JSESSIONID`). À la première visite, le servlet affiche « Welcome on my site ! » avec un formulaire qui demande le nom complet et la date de naissance. Le `doPost()` les enregistre dans la session puis redirige vers le même servlet.

```
session.setAttribute("fullName", fullName);
session.setAttribute("birthDay", birthDay);
response.sendRedirect("ShowSession");
```

Aux visites suivantes, comme les attributs existent dans la session, le servlet affiche « Welcome back ! » et calcule le nombre de jours avant l'anniversaire. Si l'anniversaire de cette année est déjà passé, on prend celui de l'année suivante.

```
LocalDate nextBirthday = MonthDay.from(birth).atYear(today.getYear());
if (!nextBirthday.isAfter(today)) {
    nextBirthday = nextBirthday.plusYears(1);
}
long days = ChronoUnit.DAYS.between(today, nextBirthday);
```

La redirection après le POST évite aussi que le navigateur renvoie le formulaire quand on actualise la page.



Figure 5 – Première visite puis message de retour.

2- Question 2 : Valeurs complexes : `ListItems`

Au lieu d'une `ArrayList`, j'ai directement stocké dans la session une `HashMap<String, Integer>` : la clé est l'élément et la valeur est le nombre de fois où il a été saisi. Comme ça, l'extra (compter les saisies) est déjà réglé. À chaque appel, on récupère la map dans la session (on la crée si elle n'existe pas), on ajoute 1 au compteur de l'élément reçu, puis on affiche la map dans un tableau.

```
HashMap<String, Integer> items =
    (HashMap<String, Integer>) session.getAttribute("items");
if (items == null) {
    items = new HashMap<String, Integer>();
}
int count = items.getOrDefault(newItem, 0);
items.put(newItem, count + 1);
session.setAttribute("items", items);
```

Quelques remarques. À la compilation, `javac` affiche une note *uses unchecked or unsafe operations* : c'est le cast de `getAttribute()` (qui renvoie un `Object`) vers `HashMap`, ce n'est pas bloquant. Une `HashMap` ne garde pas l'ordre d'ajout, donc les éléments ne s'affichent pas forcément dans l'ordre où on les a tapés. Enfin, ce servlet n'a que `doPost()` : on y revient donc avec le lien « Add another item » vers `items.html`, et pas en tapant l'adresse.



Figure 6 – Éléments saisis et leur nombre d'occurrences.

Bilan de l'exercice 1

Ce que j'ai retenu

J'ai compris la différence entre `doGet()` et `doPost()` (et pourquoi on a une erreur 405 si on appelle un servlet avec la mauvaise méthode). Un cookie est stocké chez le client et sa durée de vie se règle avec `setMaxAge()`, alors qu'une session est stockée côté serveur et le client ne garde qu'un identifiant : on peut donc y mettre des objets comme une `HashMap`, mais elle disparaît au bout d'un moment. J'ai aussi appris que les cookies doivent être ajoutés avant d'écrire la réponse, et comment compiler et déployer un servlet à la main avec `javac` et Tomcat.